

Concurrent Collection in C# with Examples

In this article, I am going to discuss the Concurrent Collection in C# with Examples. Please read our previous article where we discussed [Generic LinkedList<T> Collection Class in C#](#) with Examples.

Why do we need Concurrent Collection in C#?

In **C# 1.0**, the **System.Collections** framework was introduced and the collection classes such as **ArrayList**, **Hashtable**, **Stack**, **Queue**, etc belong to the System.Collections namespace. The problem with these collection classes is that they are not type-safe. That is they store the elements in the form of objects and because of this, we may get type mismatch exceptions, as well as performance degradation because of boxing and unboxing.

Next, in C# 2.0, the **System.Collections.Generic** Framework introduced and the collection classes **List<T>**, **Dictionary<TKey, TValue>**, **Stack<T>**, **Queue<T>**, etc are belongs to System.Collections.Generic namespace. These collection classes are Type-Safe but not Thread Safe. Typesafe means whenever we are going to declare any generic type collection, we need to specify the type that is going to be held by the generic collection. And whenever we are going to retrieve any item from the collection, we will get the actual type of item. That means boxing and unboxing are not required.

But Generic Collection Classes are not Thread Safe. So, as a developer, it is our responsibility to provide thread safety. For example, let's say we have a dictionary collection. And that dictionary collection is shared by several threads. Then we may face some concurrency issues when two or more threads try to access the dictionary collection at the same point in time, like adding/removing items from the same dictionary collection at the same time.

Example to Understand Thread Safety Problem with Generic Collections in C#:

In the below example, we have created one dictionary object with int as the key and string as the value. Then we created two methods i.e. Method1 and Method2 and both these methods are trying to add some elements to the dictionary collection. Then inside the main method, we have created two threads i.e. t1 and t2. Thread t1 points to Method1 and thread t2 points to Method2. And we then call the start method of both threads which will execute both Methods concurrently.

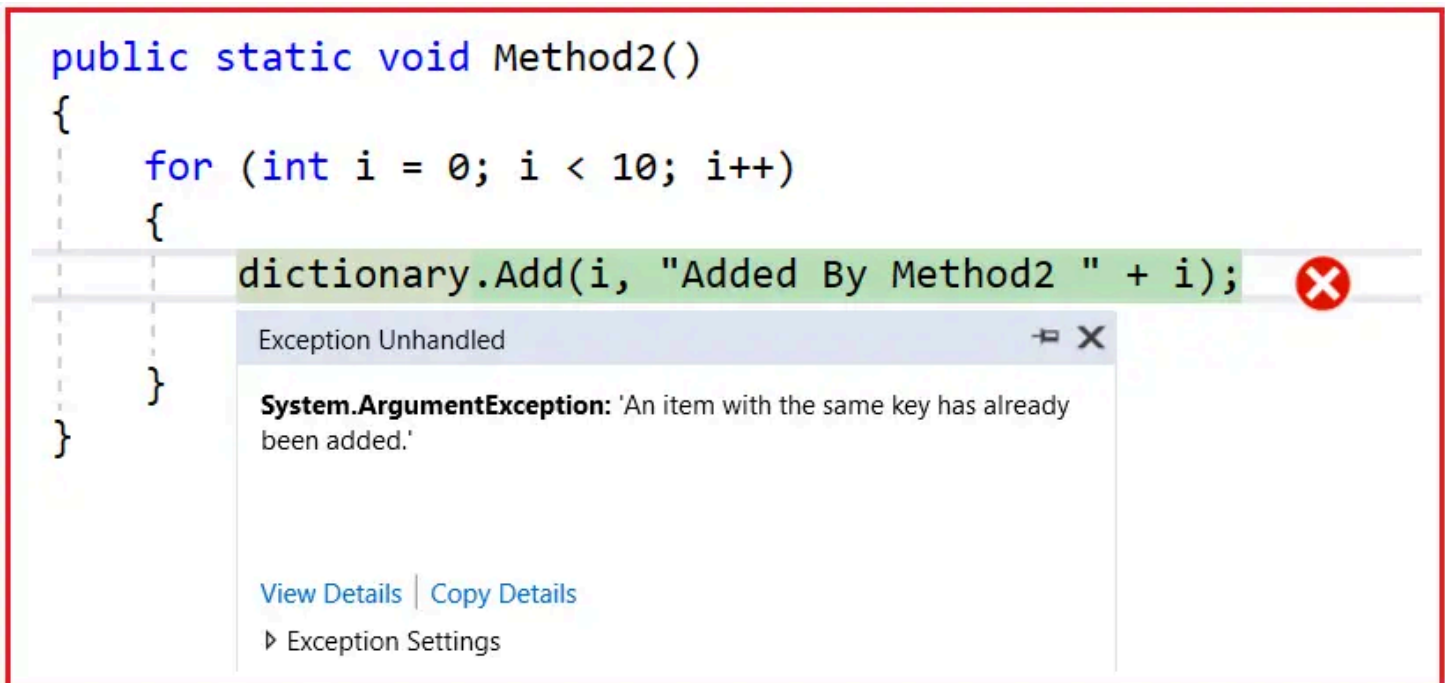
```
using System;
using System.Collections.Generic;
using System.Threading;
namespace ConcurrentCollections
{
    class Program
    {
        static Dictionary<int, string> dictionary = new Dictionary<int, string>();
        static void Main(string[] args)
        {
            Thread t1 = new Thread(Method1);
            Thread t2 = new Thread(Method2);
            t1.Start();
            t2.Start();
            Console.ReadKey();
        }
    }
}
```

```

public static void Method1()
{
    for (int i = 0; i < 10; i++)
    {
        dictionary.Add(i, "Added By Method1 " + i);
        Thread.Sleep(100);
    }
}
public static void Method2()
{
    for (int i = 0; i < 10; i++)
    {
        dictionary.Add(i, "Added By Method2 " + i);
        Thread.Sleep(100);
    }
}
}

```

Now, run the above code and after some time you will get the following error. This is because the dictionary keys must be unique and the same key already added by one of the methods. We got this error because Generic Dictionary does not provide thread safety by default.



As a developer, we can implement Thread safety with the help of locks and monitors. But locking the entire list for the sake of **Adding/Removing** an item could be a big performance issue. This is where Concurrent Collection Classes are required. Concurrent Collections can be shared across multiple threads with no explicit locks and it also increases the performance of multi-threaded operations.

Example using ConcurrentDictionary in C#:

Now, let us rewrite the previous example using `ConcurrentDictionary` and see whether we are getting any exceptions or not. The `ConcurrentDictionary` collection class belongs to the `System.Collections.Concurrent` namespace. For now, simply copy and paste the following code and run it. From our next article onward we will discuss all the **System.Collections.Concurrent** Collection Classes in detail with examples.

In the below example, we have done three changes. First, we import the **System.Collections.Concurrent** namespace. Then we use **ConcurrentDictionary** collection class instead of the `Dictionary` collection class. Finally, we use the **TryAdd** method instead of the `Add` method.

```
using System;
using System.Threading;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace ConcurrentCollections
{
    class Program
    {
        static ConcurrentDictionary<int, string> dictionary = new
ConcurrentDictionary<int, string>();
        static void Main(string[] args)
        {
            Thread t1 = new Thread(Method1);
            Thread t2 = new Thread(Method2);
            t1.Start();
            t2.Start();
            t1.Join();
            t2.Join();
            foreach (KeyValuePair<int, string> item in dictionary)
            {
                Console.WriteLine($"Key:{item.Key}, Value:{item.Value}");
            }
            Console.ReadKey();
        }
        public static void Method1()
        {
            for (int i = 0; i < 10; i++)
            {
                dictionary.TryAdd(i, "Added By Method1 " + i);
                Thread.Sleep(100);
            }
        }
        public static void Method2()
        {
            for (int i = 0; i < 10; i++)
            {
                dictionary.TryAdd(i, "Added By Method2 " + i);
                Thread.Sleep(100);
            }
        }
    }
}
```

Now, run the above code and you will not get any exceptions. If you further notice then you will see that some of the elements are added by Method1 and some of the elements are added by Method2 as shown in the below output.

```
Key:0, Value:Added By Method1 0
Key:1, Value:Added By Method2 1
Key:2, Value:Added By Method2 2
Key:3, Value:Added By Method1 3
Key:4, Value:Added By Method2 4
Key:5, Value:Added By Method1 5
Key:6, Value:Added By Method2 6
Key:7, Value:Added By Method1 7
Key:8, Value:Added By Method1 8
Key:9, Value:Added By Method1 9
```

What ConcurrentDictionary Do internally?

A thread-safe Add/Remove from the dictionary. It provides user-friendly methods that make it unnecessary for code to check if a key exists before adding/removing it.

1. **AddOrUpdate**: Adds a new entry if does not exist else updates the existing one
2. **GetOrAdd**: Retrieves an item if exists, else first adds it then retrieve it
3. **TryAdd, TryGetValue, TryUpdate, TryRemove**: Try to do the specified operation like Add/Get/Update/Remove. It returns true if performing the operation and false if failing to do the operation.

Note: We will discuss the above methods in detail once start discussing the Concurrent Collection Classes.

What are Concurrent Collections in C#?

The .NET Framework 4.0 provides new classes for concurrency as Concurrent Collections. The Concurrent Collections allow us to create type-safe (due to the generic implementation) as well as thread-safe collections.

These collection classes are specifically used in multithreading. These collections can be accessed by multiple threads at a time hence they are called Concurrent collections. They allow us to share the data between several threads with thread safety. They are available under the namespace System.Collections.Concurrent. Below are the different types of concurrent collections.

1. **ConcurrentDictionary< Key, Value>**: Thread-safe version of Generic Dictionary.
2. **ConcurrentQueue<T>**: Thread-safe version of the generic queue (FIFO Data Structure).
3. **ConcurrentStack<T>**: Thread-safe version of generic stack (LIFO Data Structure).
4. **ConcurrentBag<T>**: Thread-safe implementation of an unordered collection.
5. **BlockingCollection<T>**: Provides a Classical Producer-Consumer pattern.

Note: The producer and consumer pattern can easily be implemented while using ConcurrentStack, ConcurrentQueue, and ConcurrentBag as they implement the interface IProducerConsumerCollection.

Benefits of Concurrent Collections in C#:

1. As a developer, we don't need to worry about thread safety.
2. It uses lightweight synchronization like SpinWait, SpinLock, etc that uses spinning before putting threads to wait – for short wait periods, spinning is less expensive than waiting which involves kernel transition.
3. It provides faster add/remove/iterate in a multithreading environment without writing the code for it.
4. Some classes like ConcurrentQueue and ConcurrentStack don't rely on Interlocked operations instead of locks which makes them faster.

When to use Concurrent Collections over Generic Collections in C#?

1. The Concurrent Collections need to be used when the collections are getting changed or data is added/updated/deleted by multiple threads. If the requirement is only for reading operations in a multithreaded environment, then generic collections can be used.
2. If locking is needed at a few places, manual locking or synchronization techniques can also be used however if it is required at several places, using concurrent collection is a good choice.
3. Concurrent collections are designed to be used in cases when excessive thread safety is required, overly using manual locking can lead to deadlock and other issues.
4. Internally, Concurrent Collections use several algorithms to minimize thread blocking.

In the next article, I am going to discuss the [ConcurrentDictionary< Key, Value> Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **Concurrent Collections in C#** with Examples. I hope this Concurrent Collection in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.